

1. Datos del Proyecto

Nombre del proyecto	Sistema de Gestión de Citas Odontológicas (SIGCO)
Cliente / Solicitante	Jose Pablo Lemus Concua
Sector	Atención Odontológica
Descripción	Sistema web para gestionar citas de pacientes en una clínica odontológica. Permite registrar pacientes, agendar citas, controlar disponibilidad de doctores, visualizar historial clínico y mejorar la atención mediante organización digital centralizada.
Tipo de despliegue	Monolito (Web Service)
Plataforma	Web
Escala de usuarios	< 100 usuarios concurrentes
Presupuesto total	Q 240,000 (~USD 31,000). Infra mensual estimada: ~USD 43 / mes.
Entorno	Cloud — AWS
Base de datos	Provista por el área de BD — diagrama ER en Lucidchart
Metodología	SCRUM — Sprints de 2 semanas, 16 semanas de duración
Equipo	8 personas: QA, PM, Analista, Desarrolladores, Diseñador, DBA, Arquitecto, Seguridad

2. Requerimientos No Funcionales

Rendimiento

Tiempo de respuesta promedio	< 500 ms para operaciones CRUD estándar
Latencia máxima (p95)	< 1,500 ms
Estrategia de escalado	Vertical en primera instancia. Horizontal si se supera 80% de CPU de forma sostenida.
Crecimiento proyectado	Bajo — clínica pequeña, < 100 usuarios activos simultáneos en pico

Disponibilidad

Disponibilidad requerida	99.5% (permite ~44 h downtime/año — aceptable para clínica pequeña)
RTO	4 horas
RPO	24 horas (backup diario es suficiente dado el volumen de operaciones)
Estrategia ante fallos	Retry automático en cliente. Reinicio automático del contenedor (Docker restart policy: always).

Seguridad

Datos sensibles	Sí — datos de salud de pacientes (historial clínico, información personal)
Normativas de referencia	Buenas prácticas de protección de datos. Se toman los lineamientos de HIPAA como guía, aunque no aplica formalmente en Guatemala.
Autenticación	JWT con refresh token (expiración: 1 h / 7 días)
Autorización	RBAC — roles: Administrador, Recepcionista, Odontólogo
Encriptación en tránsito	TLS 1.2+ (HTTPS obligatorio en producción)
Encriptación en reposo	Sí — campos sensibles de pacientes cifrados en BD

3. Arquitectura de Software

Estilo arquitectónico

Patrón elegido	MVC (Model – View – Controller)
Justificación	El sistema es una aplicación web de gestión de tamaño pequeño-mediano con < 100 usuarios. MVC es el patrón más adecuado: es bien conocido por el equipo, acelera el desarrollo y evita la complejidad innecesaria de patrones como Hexagonal u Onion, que están pensados para sistemas de mayor escala o con múltiples puntos de entrada. Dadas las 16 semanas de desarrollo y el presupuesto disponible, la simplicidad de MVC es una ventaja directa.

Módulos del sistema

Módulo	Responsabilidad	Expone API	Tecnología
Gestión de Pacientes	Registro, consulta y actualización de información de pacientes	Sí	REST API
Gestión de Citas	Agendar, modificar, cancelar y reprogramar citas; validación de disponibilidad	Sí	REST API
Agenda de Odontólogos	Visualización de agenda diaria/semanal por odontólogo y especialidad	Sí	REST API
Recordatorios Automáticos	Envío de recordatorios de citas por correo electrónico vía cron programado	No	Servicio interno
Reportes Administrativos	Generación de reportes de citas atendidas, canceladas y pendientes	Sí	REST API
Autenticación	Login, gestión de sesión JWT y control de acceso por rol (RBAC)	Sí	REST API

4. Stack Tecnológico

Capa	Tecnología	Versión	Justificación
Backend / API	NestJS	10.x	Framework estructurado sobre Node.js con módulos, inyección de dependencias y validación automática. Mejor que Express puro para un sistema con múltiples módulos (citas, agenda, pacientes, reportes) y un equipo de 8 personas que necesita convenciones claras.
Frontend Web	Next.js	14	Routing integrado, layout compartido y mejor experiencia de desarrollo que React puro. Para un sistema de gestión interno con múltiples vistas (agenda, pacientes, citas, reportes), Next.js da estructura sin configuración adicional de router.

ORM	Prisma	5.x	Tipado fuerte, migraciones integradas, reduce errores de SQL manual. Funciona directamente con PostgreSQL.
Base de Datos	PostgreSQL	15	Provista por el área de BD. Relacional y robusto para datos estructurados de pacientes y citas.
Autenticación	JWT (jsonwebtoken)	9.x	Stateless, fácil de implementar con RBAC. Sin necesidad de manejo de sesiones en servidor.
Recordatorios	Nodemailer + cron	—	Envío de correos programados. Suficiente para esta escala, sin necesidad de infraestructura de colas.
Contenedores	Docker	24.x	Consistencia entre entornos. Docker Compose en producción es suficiente para una sola instancia.
CI/CD	GitHub Actions	—	Integrado con el repositorio, automatiza build, test y deploy sin costo adicional.
Servidor web	NGINX	1.25	Proxy inverso, sirve el frontend estático, termina TLS. Estable y de bajo consumo de recursos.

5. Infraestructura

Componentes de red y enrutamiento

Componente	Aplica	Herramienta / Servicio	Notas
Proxy inverso	Sí	NGINX	Enruta tráfico al backend NestJS, sirve el build de Next.js y termina TLS
Balanceador de carga	No	—	Con < 100 usuarios no se justifican. NGINX como proxy es suficiente.
API Gateway	No	—	Sistema monolítico, no hay múltiples servicios que enrutar.
CDN	No	—	Tráfico bajo. Los assets estáticos los sirve NGINX directamente.

WAF / Firewall de aplicación	No	—	Compensado con validación de inputs en backend y rate limiting.
Red privada (VPC)	Sí	AWS VPC	BD en subred privada, accesible solo desde el servidor de aplicación dentro de la VPC.

Almacenamiento

Tipo	Aplica	Servicio	Notas
Disco persistente (EBS)	Sí	AWS EBS — 20 GB gp3	Para las instancias EC2 (servidor de app y staging)
Object Storage (archivos)	No	—	El sistema no maneja carga de archivos ni imágenes en v1

Servidores y capacidad

A < 100 usuarios no se requieren clusters ni réplicas. Se trabaja con instancias únicas y backups automáticos.

Componente	Instancias	Spec / Tipo	Notas
Servidor de aplicación	1	AWS EC2 t3.small (2 vCPU / 2 GB RAM)	Corre backend NestJS en Docker + NGINX. Suficiente para la carga proyectada.
Base de Datos	1	AWS RDS db.t3.micro (PostgreSQL 15)	Instancia gestionada. Backups automáticos diarios incluidos. Multi-AZ desactivado para reducir costo.
Servidor de staging	1	AWS EC2 t3.micro	Ambiente de pruebas. Puede apagarse fuera de horario laboral.

6. Base de Datos

Diseño provisto por el área de BD. Diagrama ER disponible en Lucidchart (entregable externo).

Motor	PostgreSQL 15
--------------	---------------

Tipo	Relacional
Justificación	Las entidades del sistema (Paciente, Cita, Odontólogo, Horario, Especialidad, Usuario) tienen relaciones bien definidas. Un modelo relacional garantiza integridad referencial y es el más natural para este dominio.
Entidades principales	Paciente, Cita, Odontólogo, Especialidad, Horario, Usuario
Despliegue	AWS RDS — backups automáticos, parches gestionados, métricas integradas en CloudWatch
Backup	Snapshot diario automático (RDS). Retención: 7 días.
ORM / Migraciones	Prisma 5.x + Prisma Migrate
Acceso	Solo desde la instancia EC2 dentro de la VPC. Puerto 5432 no expuesto a internet.

7. Seguridad

Área	Implementación
Autenticación	JWT con expiración de 1 hora + refresh token de 7 días almacenado en cookie httpOnly
Autorización	RBAC — 3 roles: Administrador (acceso total), Recepcionista (pacientes y citas), Odontólogo (solo su agenda y citas asignadas)
Contraseñas	Hash con bcrypt (salt rounds: 12)
Gestión de secretos	Variables de entorno en el servidor (.env). En producción: AWS Secrets Manager para credenciales de BD y claves JWT.
Rate limiting	express-rate-limit — máx. 100 peticiones / 15 min por IP en endpoints de autenticación
CORS	Solo orígenes autorizados (dominio del frontend en producción). Deshabilitado en rutas de API internas.
Validación de inputs	express-validator en todos los endpoints con datos de entrada
HTTPS	TLS 1.2+ — certificado gestionado con AWS Certificate Manager (ACM)
Aislamiento de BD	RDS en subred privada de la VPC. Sin acceso directo desde internet.
Logs de auditoría	Registro de acciones sensibles (login, cambio de cita, acceso a historial) en tabla de auditoría en BD

8. Contenedores y Despliegue

Estrategia de contenedores	Docker en todos los entornos. Docker Compose en producción (suficiente para instancia única).
Registry de imágenes	AWS ECR (Elastic Container Registry)
Proxy / Servidor web	NGINX — termina TLS, redirige /api/* al contenedor NestJS. Next.js puede servirse desde el mismo EC2 o desde Vercel.
Health checks	GET /health → 200 OK (liveness). GET /ready → verifica conexión a BD y estado del servicio (readiness).
Estrategia de deploy	GitHub Actions → build + push a ECR → SSH al servidor EC2 → docker compose pull && up -d
Ambientes	local → staging (EC2 t3. micro) → production (EC2 t3.small)
Rollback	Mantener imagen anterior en ECR. Si el smoke test falla, se hace pull de la imagen anterior y up -d.

9. CI/CD

Plataforma	GitHub
Estrategia de branching	GitHub Flow — main (producción), develop (integración), feature/* (nuevas funcionalidades)
Herramienta CI/CD	GitHub Actions
Etapas del pipeline	1. Lint (ESLint) → 2. Tests unitarios (Jest) → 3. Build Docker image → 4. Push a ECR → 5. Deploy a EC2 via SSH → 6. Smoke test (curl /health)
Triggers	Push a develop → deploy automático a staging. PR mergeado a main → deploy a producción con aprobación manual (environment protection).
Convención de commits	Conventional Commits (feat;, fix;, chore;, docs;)

10. Observabilidad

Configuración mínima adecuada para < 100 usuarios. Sin herramientas de pago adicionales.

Área	Aplica	Herramienta	Descripción
Logging	Sí	Winston + AWS CloudWatch Logs	Logs estructurados en JSON. Captura errores, requests y eventos de auth. Retención: 30 días.
Métricas de infra	Sí	AWS CloudWatch Metrics	CPU, memoria, disco y conexiones de BD (EC2 + RDS). Incluido sin costo adicional.
Alertas	Sí	AWS CloudWatch Alarms	Alerta si CPU > 80% por 5 min consecutivos, o si el health check falla 3 veces seguidas. Notificación por correo (SNS).
Tracing distribuido	No	—	No aplica — sistema monolítico. No hay servicios distribuidos que trazar.
Dashboards	No	—	CloudWatch básico es suficiente a esta escala.

11. Costos Estimados de Infraestructura

Componente	Servicio	Tier / Tipo	USD / mes	Notas
Servidor de aplicación	AWS EC2	t3.small (2 vCPU / 2 GB)	~\$17	On-demand. Reducible a ~\$12 con Reserved Instance.
Base de Datos	AWS RDS	db.t3.micro PostgreSQL 15	~\$15	Backups automáticos incluidos
Servidor de staging	AWS EC2	t3.micro	~\$8	Apagar fuera de horario laboral para reducir costo
Almacenamiento	AWS EBS	20 GB gp3 x2 instancias	~\$2	Costo fijo por volumen

Red / Transferencia	AWS	< 10 GB / mes estimado	~\$1	<i>Tráfico bajo dado el volumen de usuarios</i>
Registry de imágenes	AWS ECR	< 1 GB de imágenes	~\$0.10	<i>Primeros 500 MB gratuitos</i>
Monitoring / Logs	CloudWatch	Basic monitoring	\$0	<i>Incluido en EC2 y RDS sin costo adicional</i>
Certificado SSL	AWS ACM	Certificado gestionado	\$0	<i>Gratuito con AWS ACM</i>
Total mensual estimado			~\$43	<i>prod + staging</i>

Presupuesto total del proyecto	Q 240,000 (~USD 31,000)
Infra mensual estimada	~USD 43 / mes (producción + staging)
Infra por 16 semanas (4 meses)	~USD 172 — menos del 1% del presupuesto total
Observación	El grueso del presupuesto corresponde al equipo de desarrollo (8 personas, ~USD 32,000 en horas).

13. Decisiones Arquitectónicas

ID	Área	Opción Elegida	Alternativa	Justificación
ADR-01	Patrón arquitectónico	MVC	Hexagonal / Clean Architecture	Sistema pequeño de una sola plataforma web. MVC es suficiente, conocido por el equipo y más rápido de implementar en 16 semanas.
ADR-02	Framework backend	NestJS	Express puro, Go/Fiber	NestJS estructura el proyecto con módulos y DI desde el inicio. A esta escala (< 100 usuarios) el rendimiento de Express o Go no aporta ventaja real. La estructura de NestJS reduce errores y acelera el desarrollo en un equipo de 8.
ADR-02b	Framework frontend	Next.js	React puro, Vue	Next.js da routing integrado y layouts compartidos sin configurar React Router. Para un sistema con múltiples vistas (agenda, pacientes, citas), la estructura de Next.js acelera el desarrollo.
ADR-03	ORM	Prisma	Sequelize, TypeORM	Tipado fuerte y migraciones integradas. Reduce errores de SQL manual y acelera el desarrollo sin sacrificar flexibilidad.
ADR-04	Despliegue en cloud	AWS (EC2 + RDS)	GCP, Azure, VPS propio	Mejor relación costo-beneficio a esta escala. RDS gestiona la BD automáticamente. El equipo tiene más referencias y documentación en AWS.
ADR-05	Sin balanceador de carga	NGINX como proxy único	AWS ALB, HAProxy	Con < 100 usuarios concurrentes, una sola instancia EC2 t3.small es suficiente. Un ALB costaría ~\$16/mes adicionales sin beneficio real.

ADR-06	Recordatorios con cron	Nodemailer + node-cron	RabbitMQ, SQS, Kafka	Un sistema de colas es innecesario a esta escala. Un cron interno es más simple de mantener y suficiente para envíos programados de recordatorios.
--------	------------------------	------------------------	----------------------	--

12. Diagrama de Arquitectura

